



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems
Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN

logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`;
ELSE `move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

Because a table driven agent needs to store an entry for every percept sequence meaning the domain is infinite. So no hardware can hold this table. As the lifetime increases the table grows exponentially.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify

and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The conditional rules for the SimpleReflexAgent are given below:

```
def sense_and_act(self, percept: dict) -> str:
    if percept['food_here']:      # IF food here
        return 'Suck'           # THEN eat it
```

The above 2 lines says that there are food on the current square and action as suck. According to the Condition-Action-Rules it says,

IF food_here is TRUE THEN suck

```
    if percept['wall_ahead']:    # IF blocked in front
        return 'TurnLeft'       # THEN turn left
```

The above 2 lines tell the agent that if there is a wall directly in front of agent the turn left. According to the Condition-Action-Rules it says,

IF wall_head is TRUE THEN TurnLeft

```
    return 'Forward'           # ELSE keep going
```

The above line tell the agent if neither is true go forward.
ELSE THEN go Forward

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

SimpleReflexAgent looped because it went around the perimeter of the grid endlessly, running Forward to a boundary, firing TurnLeft, and returning to its exact starting state at (0,0) facing Right after about 48 steps — so it never entered the interior and the run ended on the 60-step cap rather than by clearing the food. Partial observability caused this because `get_percept()` hides all global information and gives the agent only the booleans `food_here` and `wall_ahead`, collapsing 576 distinct world states (12×12 cells $\times$ 4 facings) into just 4 perceptual categories, so a corner it has visited ten times looks identical to one it has never seen. The lack of percept history compounds this fatally: the agent has no `__init__` and no internal state, so it implements $f : P \rightarrow A$ instead of $f : P^* \rightarrow A$, meaning the information missing from the current percept cannot be recovered from the past either. With a stateless deterministic policy driving a deterministic finite world, repetition is mathematically guaranteed by the pigeonhole principle — which is exactly why the `ModelBasedAgent` fixes it by storing `visited_cells` and testing ahead not in `self.visited_cells`, restoring the percept history the reflex agent throws away.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Terminology note: the transition model is how the world evolves (including as a result of actions); the sensor model is how the world state shows up in percepts — the question's labels are the reverse way round.

Transition Model — agent.py lines 64–70. Since `get_percept()` hides all coordinates, the agent uses dead reckoning: it remembers `self.last_action` (line 82) and applies its own copy of the environment's physics — Forward shifts `self.pos` via `DELTA`s, and TurnLeft/TurnRight rotate `self.heading` through `TURN_ORDER` with modulo-4 arithmetic, exactly mirroring `execute_action`. This correctly predicts where it must now be, but it assumes Forward always succeeds (a blocked move would silently corrupt the belief, prevented only by the `not percept['wall_ahead']` guard at line 91), it trusts the hard-coded starting heading 'Right', and it models only the agent's own actions — the randomly moving opponents are not represented at all.

Sensor Model — lines 72–78. This grounds the raw booleans in the map: `self.visited_cells.add(self.pos)` records coverage, and `wall_ahead` — a percept carrying no location — is bound to the concrete cell ahead computed from the believed position and heading, with discard clearing stale beliefs. Together these two sets are a compressed summary of the whole percept history, which is what lets the rules test ahead not in `self.visited_cells` (line 91) — a condition that exists nowhere in the percept — and thereby break the reflex agent's loop. Its weakness is sparsity: one cell per step, unfaced cells optimistically assumed open, and the `toxin_here` and `opponent_ahead` percepts recorded nowhere.

The sequencing is right (predict → update → decide), and the models are accurate for what they cover; the limitation is scope, not correctness — they capture static terrain and the agent's own deterministic actions, but not the environment's stochastic elements.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING